

Loop Elimination in Process Graphs is Confluent when Pruning Steps are Added

Clemens Grabmayer

<https://clegra.github.io>

Department of Computer Science



L'Aquila, Italy

IWC 2026

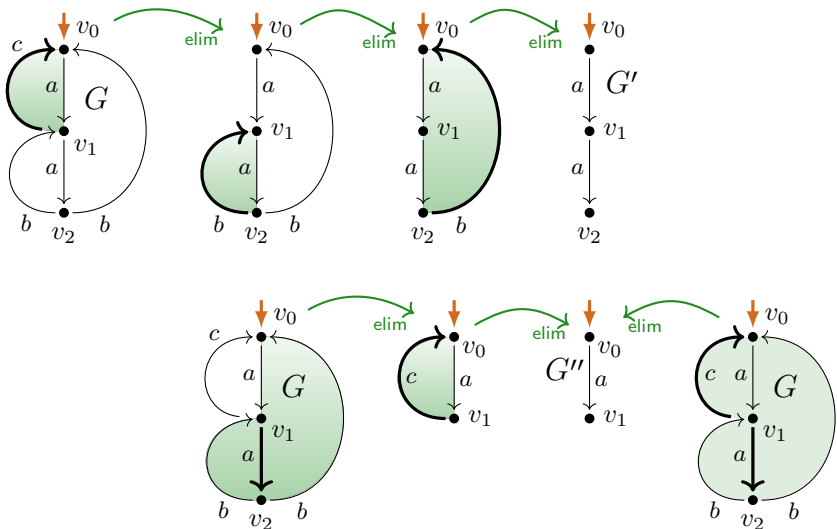
Lisbon

July 24, 2026

Overview

- ▶ Loop elimination $\rightarrow_{\text{elim}}$ in process graphs
 - ▶ Milner's process interpretation P of regular expressions
 - ▶ Loop existence and elimination property LEE
 - $\text{LEE} \hat{=}$ interpretations of $(*/\perp)$ reg. expr's
 - ▶ $\rightarrow_{\text{elim}}$ is **not** confluent
- ▶ **Completion** of loop elimination $\rightarrow_{\text{elim}}$ with **pruning steps** $\rightarrow_{\text{prune}}$
 - ▶ 'critical pair' lemma for 'bi-loop' elimination steps
 - ▶ use of decreasing diagrams
- ▶ **Application 1:** LEE is decidable in polynomial time
- ▶ **Application 2:** Layered LEE = LEE
- ▶ Further questions

Loop elimination $\rightarrow_{\text{elim}}$



Process interpretation P of regular expressions *(Milner, 1984)*

$0 \xrightarrow{P} \text{deadlock } \delta, \text{ no observable behaviour}$

$a \xrightarrow{P} \text{atomic action } a, \text{ then terminate}$

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P} \text{deadlock } \delta, \text{ no observable behaviour}$

$a \xrightarrow{P} \text{atomic action } a, \text{ then terminate}$

$e_1 + e_2 \xrightarrow{P} (\text{choice}) \text{ execute } P(e_1) \text{ or } P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P} (\text{sequentialization}) \text{ execute } P(e_1), \text{ then } P(e_2)$

$e^* \xrightarrow{P} (\text{iteration}) \text{ repeat (terminate or execute } P(e))$

Process interpretation P of regular expressions (Milner, 1984)

$0 \xrightarrow{P}$ deadlock δ , no observable behaviour

$0^* \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

Process semantics $\llbracket \cdot \rrbracket_P$ of regular expressions *(Milner, 1984)*

$0 \xrightarrow{P}$ deadlock δ , no observable behaviour

$1 \xrightarrow{P}$ empty-step process ϵ , then terminate

$a \xrightarrow{P}$ atomic action a , then terminate

$e_1 + e_2 \xrightarrow{P}$ (*choice*) execute $P(e_1)$ or $P(e_2)$

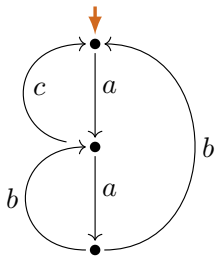
$e_1 \cdot e_2 \xrightarrow{P}$ (*sequentialization*) execute $P(e_1)$, then $P(e_2)$

$e^* \xrightarrow{P}$ (*iteration*) repeat (terminate or execute $P(e)$)

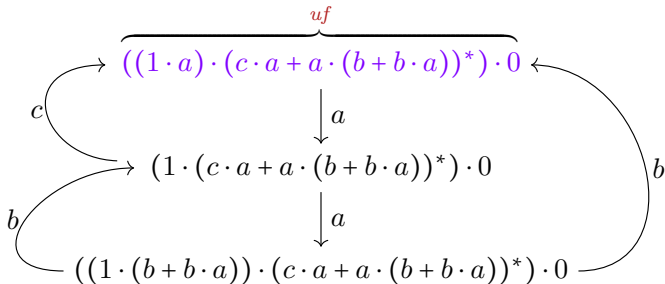
$\llbracket e \rrbracket_P := \llbracket P(e) \rrbracket_{\leftrightarrow}$ (bisimilarity equivalence class of process $P(e)$)

Process interpretation P (example)

G

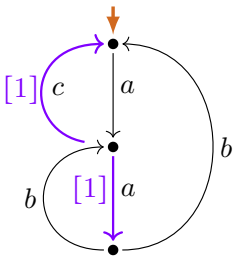


$P(uf) \simeq G$

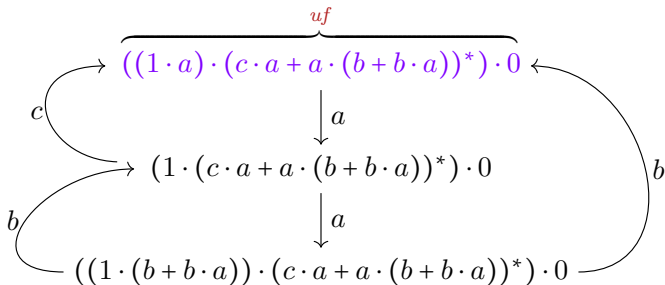


Process interpretation P (example) has property LEE

G



$P(uf) \simeq G$



Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

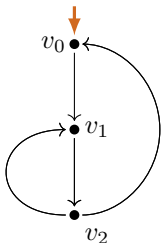
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to **it**.
- (L3) Immediate termination is **only** possible at the **start vertex**.

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.

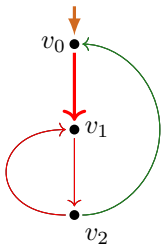


Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



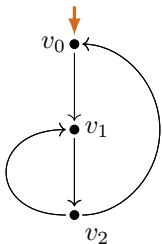
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



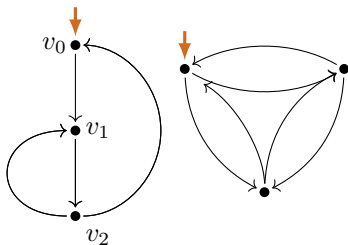
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



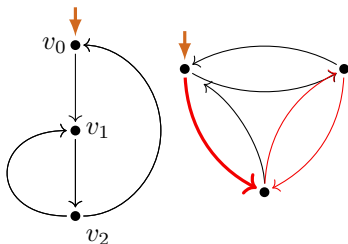
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



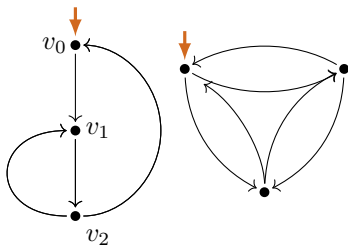
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



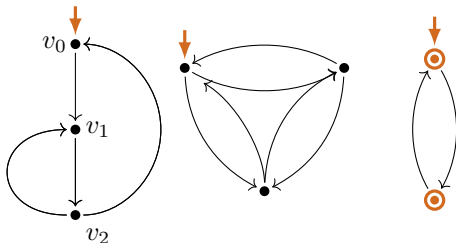
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



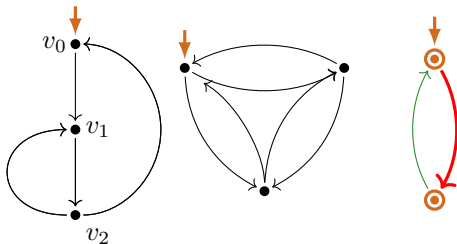
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



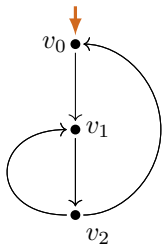
(L1), ~~(L2)~~

Loop graphs (interpretations of innermost iterations without 1)

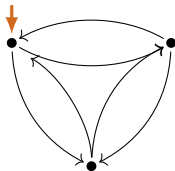
Definition

A process graph is a **loop graph** if:

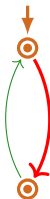
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

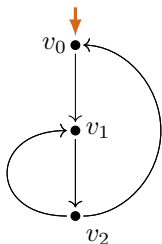


Loop graphs (interpretations of innermost iterations without 1)

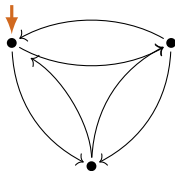
Definition

A process graph is a **loop graph** if:

- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

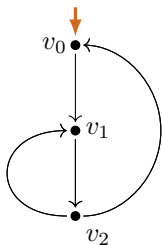


Loop graphs (interpretations of innermost iterations without 1)

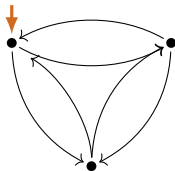
Definition

A process graph is a **loop graph** if:

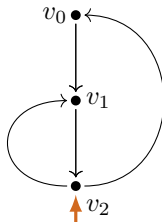
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

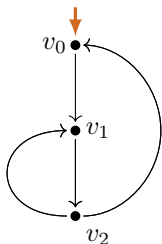


Loop graphs (interpretations of innermost iterations without 1)

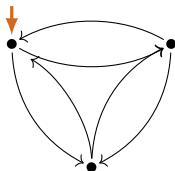
Definition

A process graph is a **loop graph** if:

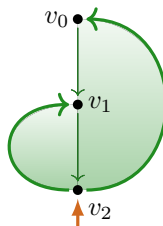
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~

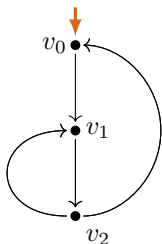


Loop graphs (interpretations of innermost iterations without 1)

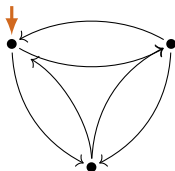
Definition

A process graph is a **loop graph** if:

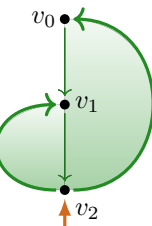
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



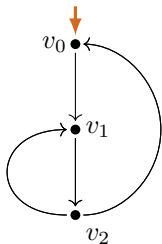
loop chart

Loop graphs (interpretations of innermost iterations without 1)

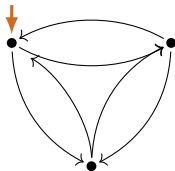
Definition

A process graph is a **loop graph** if:

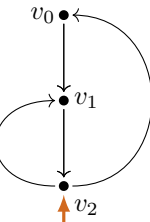
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



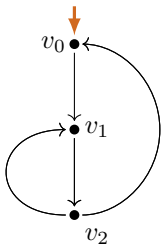
loop chart

Loop graphs (interpretations of innermost iterations without 1)

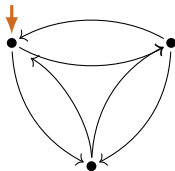
Definition

A process graph is a **loop graph** if:

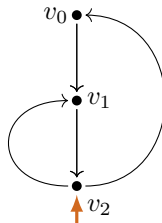
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



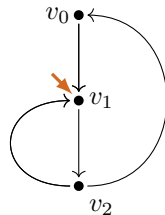
(L1), (L2)



(L1), (L2), (L3)



loop chart

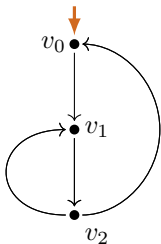


Loop graphs (interpretations of innermost iterations without 1)

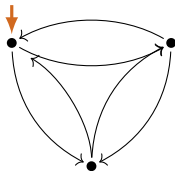
Definition

A process graph is a **loop graph** if:

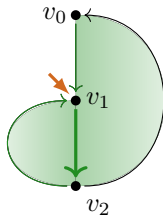
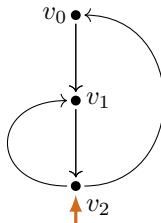
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~ **loop chart**

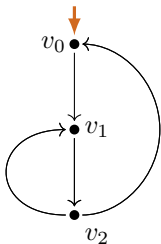


Loop graphs (interpretations of innermost iterations without 1)

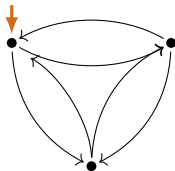
Definition

A process graph is a **loop graph** if:

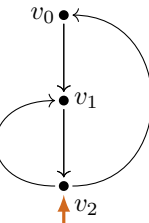
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



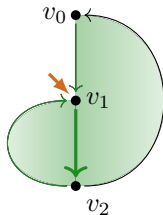
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



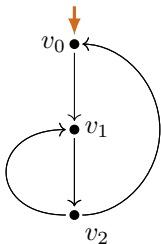
loop chart

Loop graphs (interpretations of innermost iterations without 1)

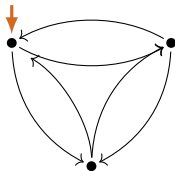
Definition

A process graph is a **loop graph** if:

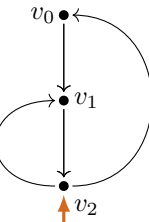
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



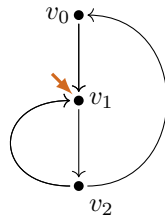
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



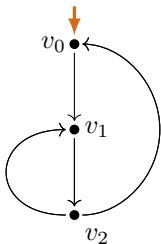
loop chart

Loop graphs (interpretations of innermost iterations without 1)

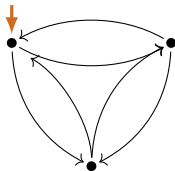
Definition

A process graph is a **loop graph** if:

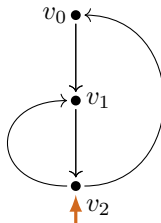
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



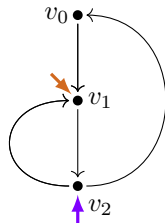
(L1), (L2)



(L1), (L2), (L3)



loop chart

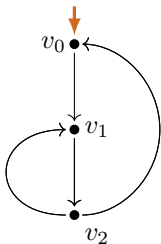


Loop graphs (interpretations of innermost iterations without 1)

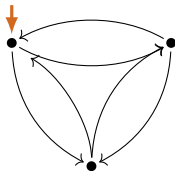
Definition

A process graph is a **loop graph** if:

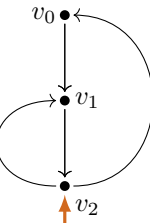
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



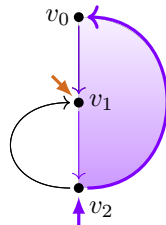
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart

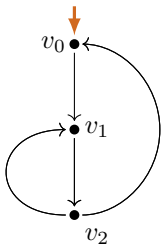


Loop graphs (interpretations of innermost iterations without 1)

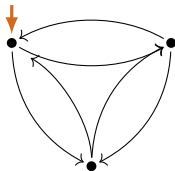
Definition

A process graph is a **loop graph** if:

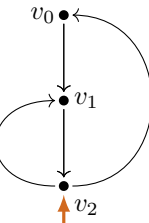
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



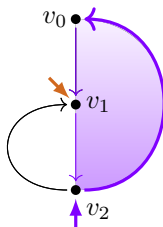
(L1), ~~(L2)~~



(L1), (L2), ~~(L3)~~



loop chart



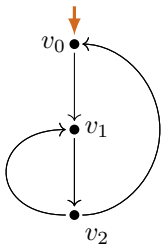
loop subchart

Loop graphs (interpretations of innermost iterations without 1)

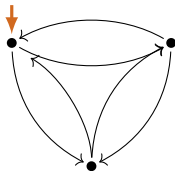
Definition

A process graph is a **loop graph** if:

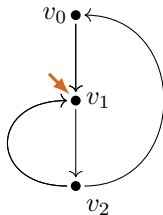
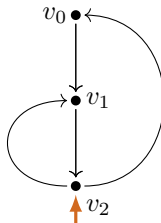
- (L1) There is an infinite path from the **start vertex**.
- (L2) Every infinite path from the **start vertex** returns to it.
- (L3) Immediate termination is **only** possible at the **start vertex**.



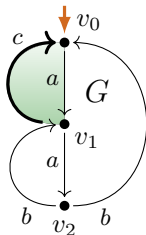
(L1), (L2)



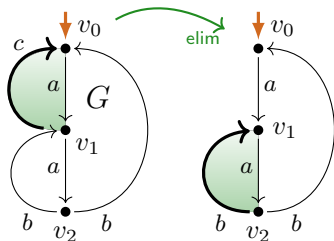
(L1), (L2), (L3) **loop chart**



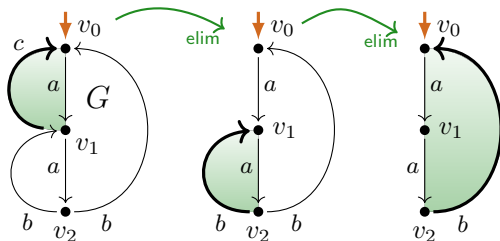
Loop elimination $\rightarrow_{\text{elim}}$



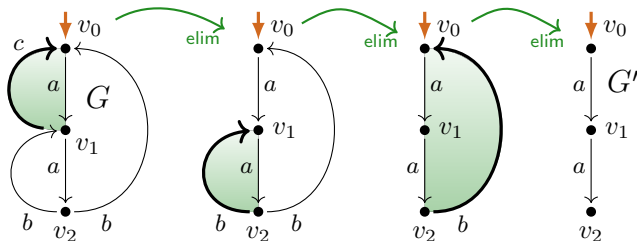
Loop elimination $\rightarrow_{\text{elim}}$



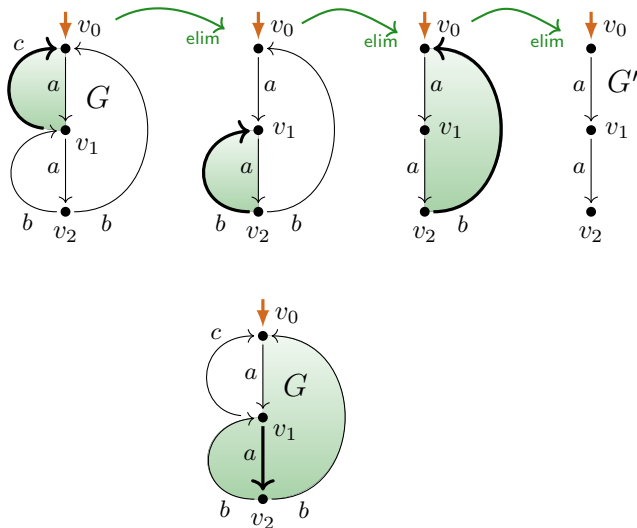
Loop elimination $\rightarrow_{\text{elim}}$



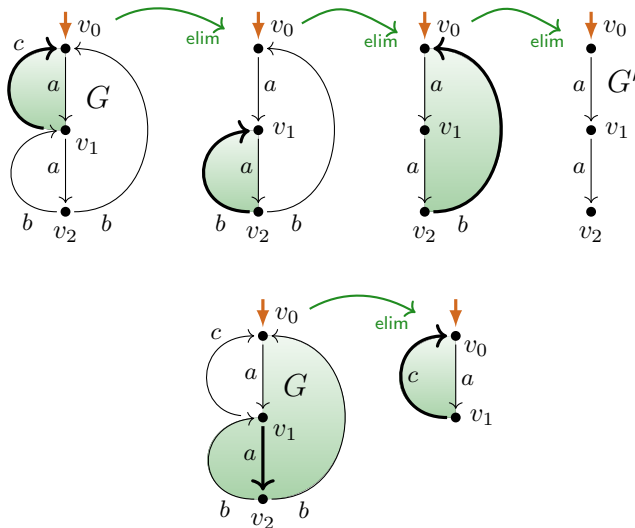
Loop elimination $\rightarrow_{\text{elim}}$



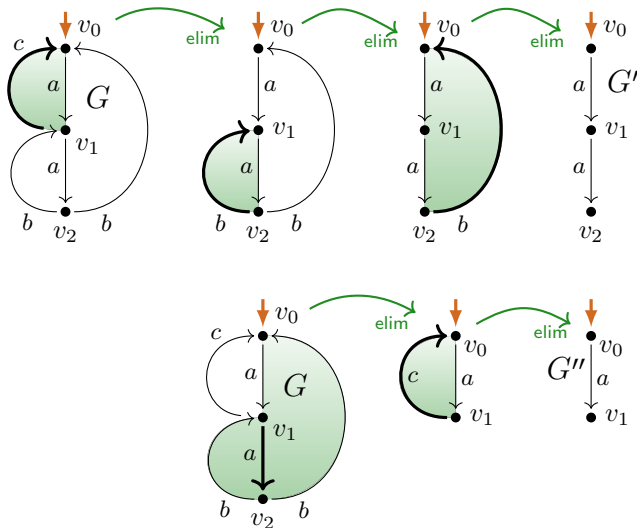
Loop elimination $\rightarrow_{\text{elim}}$



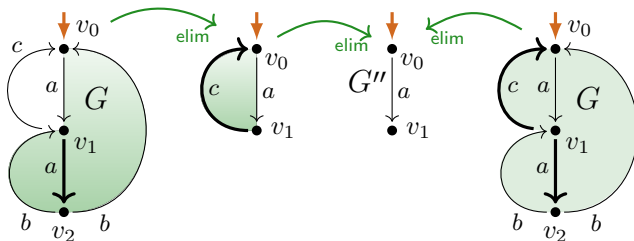
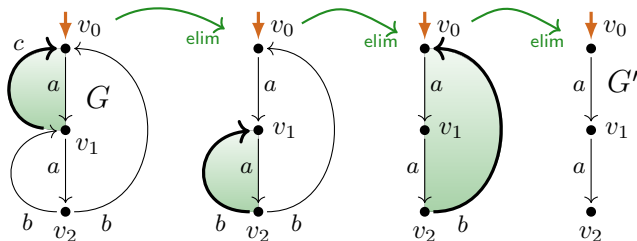
Loop elimination $\rightarrow_{\text{elim}}$



Loop elimination $\rightarrow_{\text{elim}}$



Loop elimination $\rightarrow_{\text{elim}}$



L EE

Definition

A graph G satisfies **L EE** (*loop existence and elimination*) if:

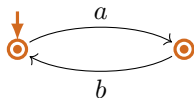
$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\xrightarrow{\text{elim}} \wedge \underbrace{\neg \infty(G_0)}_{G_0 \text{ permits no infinite trace}} \right).$$

LEE

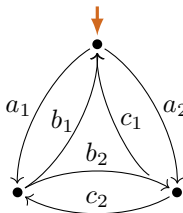
Definition

A graph G satisfies **LEE** (*loop existence and elimination*) if:

$$\exists G_0 \left(G \xrightarrow{*}_{\text{elim}} G_0 \not\xrightarrow{\text{elim}} \wedge \underbrace{\neg \infty(G_0)}_{G_0 \text{ permits no infinite trace}} \right).$$

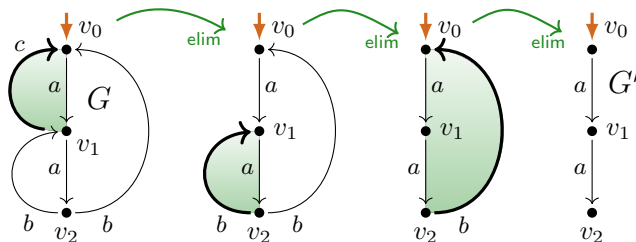


not LEE

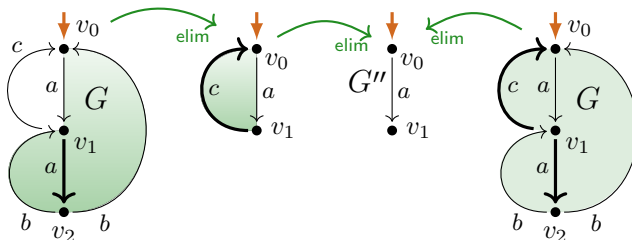


not LEE

Loop elimination $\rightarrow_{\text{elim}}$



$\text{LEE}(G)$



LEE $\stackrel{\wedge}{=}$ P -/ $P^\bullet$ -expressible graphs for $(*/\perp)$ reg. expr's

Lemma (*G-Fokkink, LICS'20*)

For all process graphs G :

$$\text{LEE}(G) \implies \exists uf \in \text{RExp}^{(*/\perp)} \left(\underbrace{G \leftrightarrow P(uf)}_{G \in \llbracket uf \rrbracket_P} \right),$$

$$\text{LEE}(G) \longleftarrow \exists uf \in \text{RExp}^{(*/\perp)} \left(G \simeq P(uf) \right).$$

$L\!E\!E \stackrel{\wedge}{=} P\text{-}/P^\bullet\text{-expressible graphs for } (*/\pm) \text{ reg. expr's}$

Lemma (*G-Fokkink, LICS'20*)

For all process graphs G :

$$L\!E\!E(G) \implies \exists uf \in RExp^{(*/\pm)} \left(\underbrace{G \rightleftharpoons P(uf)}_{G \in \llbracket uf \rrbracket_P} \right),$$

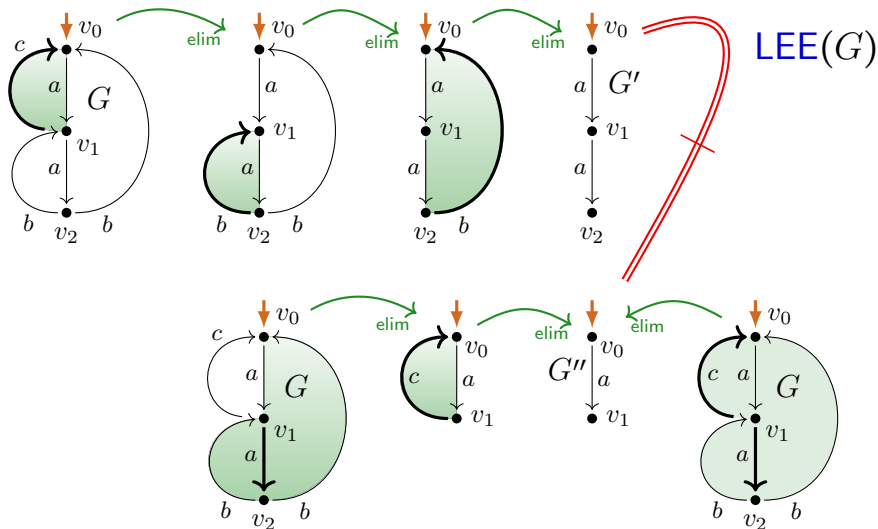
$$L\!E\!E(G) \longleftarrow \exists uf \in RExp^{(*/\pm)} \left(G \simeq P(uf) \right).$$

Theorem (*TERMGRAPH '26, LEE = LLEE from IWC'26*)

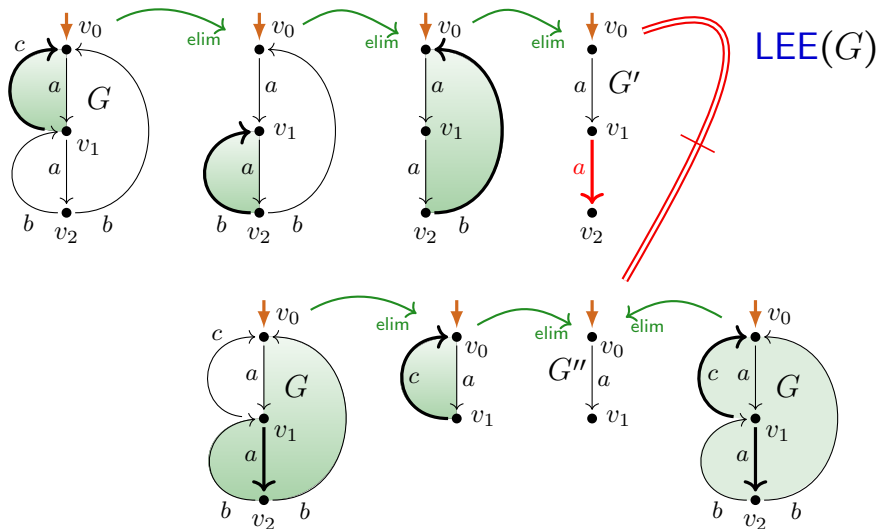
$L\!E\!E \stackrel{\wedge}{=} \text{compact}$ process interpretations $P^\bullet(uf)$ of $(*/\pm)$ reg. expr's uf :

$$L\!E\!E(G) \iff \exists uf \in RExp^{(*/\pm)} \left(G \simeq P^\bullet(uf) \right).$$

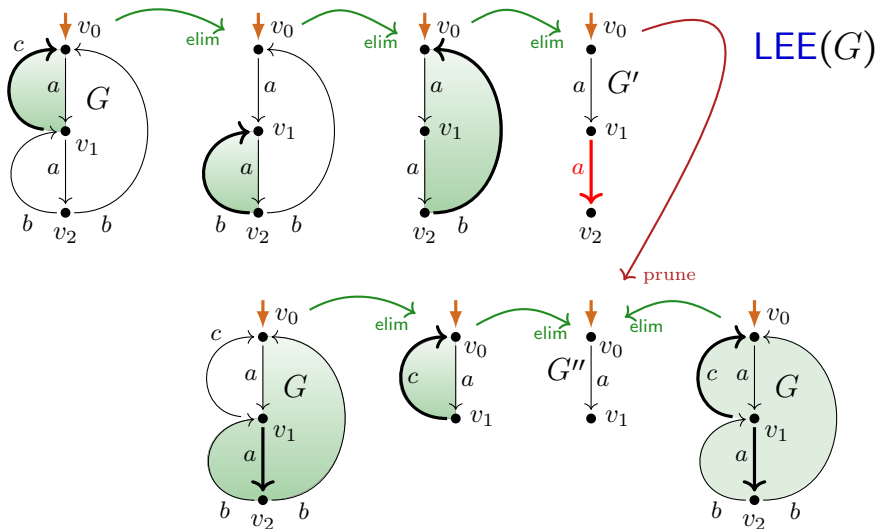
Loop elimination $\rightarrow_{\text{elim}}$ is **not** confluent



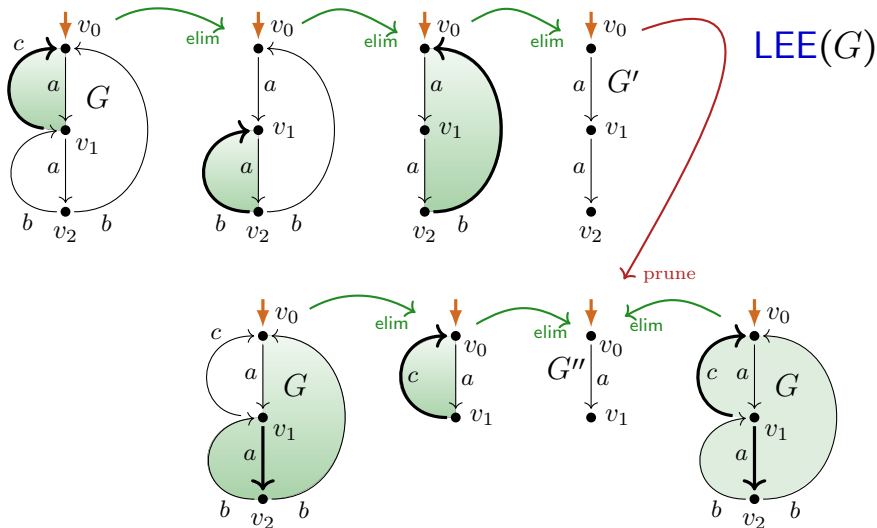
Loop elimination $\rightarrow_{\text{elim}}$ is **not** confluent



Joining an $\rightarrow_{\text{elim}}$ fork with a pruning $\rightarrow_{\text{prune}}$ step



Joining an $\rightarrow_{\text{elim}}$ fork with a pruning $\rightarrow_{\text{prune}}$ step



Overview

- ▶ Loop elimination $\rightarrow_{\text{elim}}$ in process graphs
 - ▶ Milner's process interpretation P of regular expressions
 - ▶ Loop existence and elimination property LEE
 - $\text{LEE} \hat{=}$ interpretations of $(*/\perp)$ reg. expr's
 - ▶ $\rightarrow_{\text{elim}}$ is **not** confluent
- ▶ **Completion** of loop elimination $\rightarrow_{\text{elim}}$ with **pruning steps** $\rightarrow_{\text{prune}}$
 - ▶ 'critical pair' lemma for 'bi-loop' elimination steps
 - ▶ use of decreasing diagrams
- ▶ **Application 1:** LEE is decidable in polynomial time
- ▶ **Application 2:** Layered LEE = LEE
- ▶ Further questions

'Critical Pair Lemma'

Lemma (joining $\rightarrow_{\text{elim}}$ forks)

Forks $G_2 \xleftarrow{\text{elim}} G \xrightarrow{\text{elim}} G_1$ in which loop subgraphs $G \downarrow_*^{v_2, T_2}$, $G \downarrow_*^{v_1, T_1}$ of G are eliminated, can be joined with $\rightarrow_{\text{prune}}$ steps in the three situations:

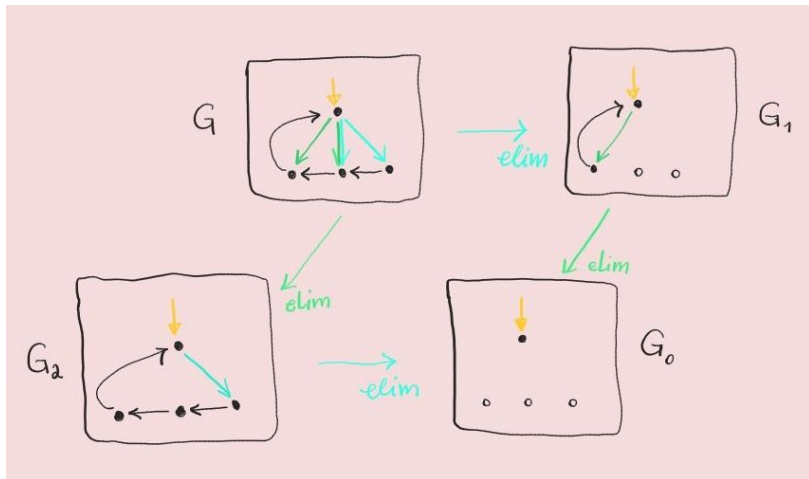
'Critical Pair Lemma'

Lemma (joining $\rightarrow_{\text{elim}}$ forks)

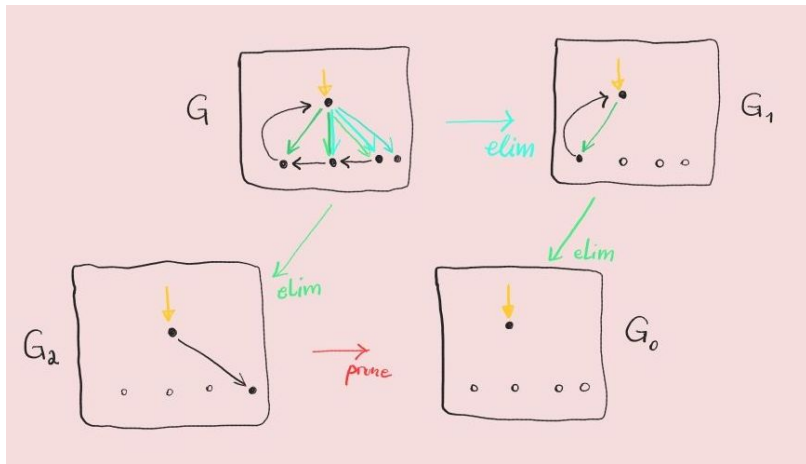
Forks $G_2 \xleftarrow{\text{elim}} G \xrightarrow{\text{elim}} G_1$ in which loop subgraphs $G \downarrow_*^{v_2, T_2}$, $G \downarrow_*^{v_1, T_1}$ of G are eliminated, can be joined with $\rightarrow_{\text{prune}}$ steps in the three situations:

- (i) **Root-overlap**: $v_1 = v_2$, and the loops form bigger loop $G \downarrow_*^{v_1, T_1 \cup T_2}$.
The $\rightarrow_{\text{elim}}$ steps may overlap in their loop-entry transitions, but the fork can always be joined by eliminating residual loop-entries as follows: $G_2 \xrightarrow{\text{elim}} \cdot \xrightarrow{*}_{\text{prune}} G_0 \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{\text{elim}} G_1$.

Root-Overlapping loop subgraphs (example a)



Root-Overlap of loop subgraphs (example b)



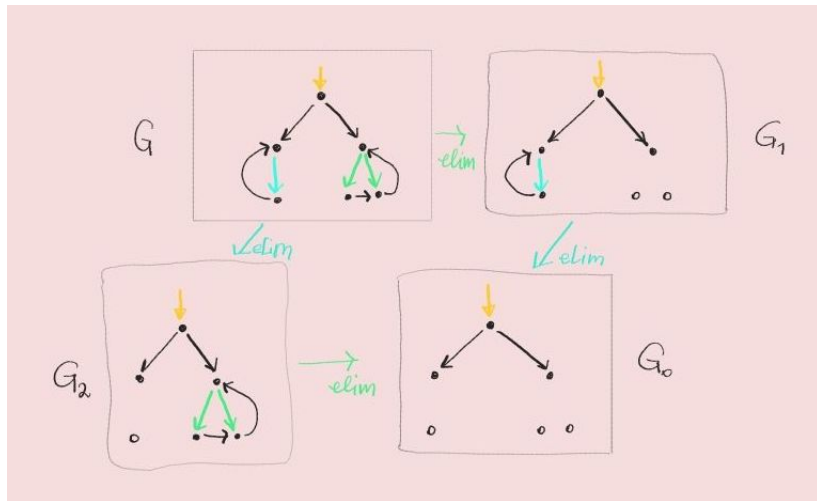
'Critical Pair Lemma'

Lemma (joining $\rightarrow_{\text{elim}}$ forks)

Forks $G_2 \xleftarrow{\text{elim}} G \xrightarrow{\text{elim}} G_1$ in which loop subgraphs $G \downarrow_*^{v_2, T_2}$, $G \downarrow_*^{v_1, T_1}$ of G are eliminated, can be joined with $\rightarrow_{\text{prune}}$ steps in the three situations:

- (i) **Root-overlap**: $v_1 = v_2$, and the loops form bigger loop $G \downarrow_*^{v_1, T_1 \cup T_2}$.
The $\rightarrow_{\text{elim}}$ steps may overlap in their loop-entry transitions, but the fork can always be joined by eliminating residual loop-entries as follows: $G_2 \xrightarrow{\text{elim}} \cdot \xrightarrow{\text{prune}}^* G_0 \xleftarrow{\text{prune}}^* \cdot \xleftarrow{\text{elim}} G_1$.
- (ii) **Parallel/nested**: $v_1 \neq v_2$, & the cyclic parts $\bigcirc(G \downarrow_*^{v_1, T_1})$, $\bigcirc(G \downarrow_*^{v_2, T_2})$ of the loops are vertex-disjoint. Then the $\rightarrow_{\text{elim}}$ steps are independent of each other, except that the elimination of one loop may eliminate the other in the garbage collection operation. The fork can always be joined as follows: $G_2 \xrightarrow{\text{elim}} G_0 \xleftarrow{\text{elim}} G_1$.

Parallel loop subgraphs (example)



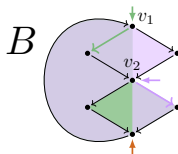
'Critical Pair Lemma'

Lemma (joining $\rightarrow_{\text{elim}}$ forks)

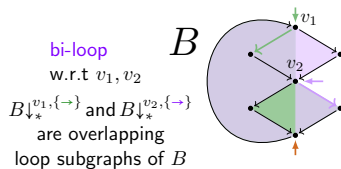
Forks $G_2 \leftarrow_{\text{elim}} G \rightarrow_{\text{elim}} G_1$ in which loop subgraphs $G_{\downarrow_*}^{v_2, T_2}$, $G_{\downarrow_*}^{v_1, T_1}$ of G are eliminated, can be joined with $\rightarrow_{\text{prune}}$ steps in the three situations:

- (i) **Root-overlap**: $v_1 = v_2$, and the loops form bigger loop $G_{\downarrow_*}^{v_1, T_1 \cup T_2}$.
The $\rightarrow_{\text{elim}}$ steps may overlap in their loop-entry transitions, but the fork can always be joined by eliminating residual loop-entries as follows: $G_2 \xrightarrow{=}_{\text{elim}} \cdot \xrightarrow{*}_{\text{prune}} G_0 \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=}_{\text{elim}} G_1$.
- (ii) **Parallel/nested**: $v_1 \neq v_2$, and the cyclic parts $\bigcirc(G_{\downarrow_*}^{v_1, T_1})$, $\bigcirc(G_{\downarrow_*}^{v_2, T_2})$ of the loops are vertex-disjoint. Then the $\rightarrow_{\text{elim}}$ steps are independent of each other, except that the elimination of one loop may eliminate the other in the garbage collection operation. The fork can always be joined as follows: $G_2 \xrightarrow{=}_{\text{elim}} G_0 \xleftarrow{=}_{\text{elim}} G_1$.
- (iii) **Non-root overlap**: $v_1 \neq v_2$, the start-vertex connected cyclic parts $\bigcirc(G_{\downarrow_*}^{v_1, T_1})$, and $\bigcirc(G_{\downarrow_*}^{v_2, T_2})$ have common vertices. The two loops together form a **bi-loop** w.r.t. v_1, v_2 . The fork can always be joined as follows: $G_2 \xrightarrow{=}_{\text{elim}} \cdot \xrightarrow{*}_{\text{prune}} G_0 \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=}_{\text{elim}} G_1$.

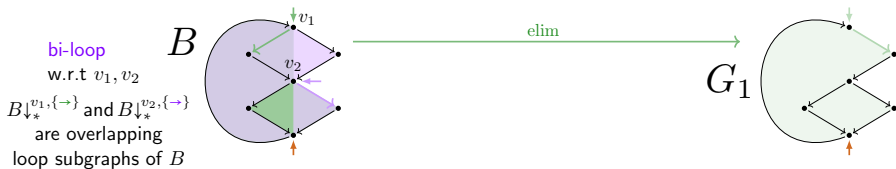
Non-Root overlap / 'critical pair' (example)



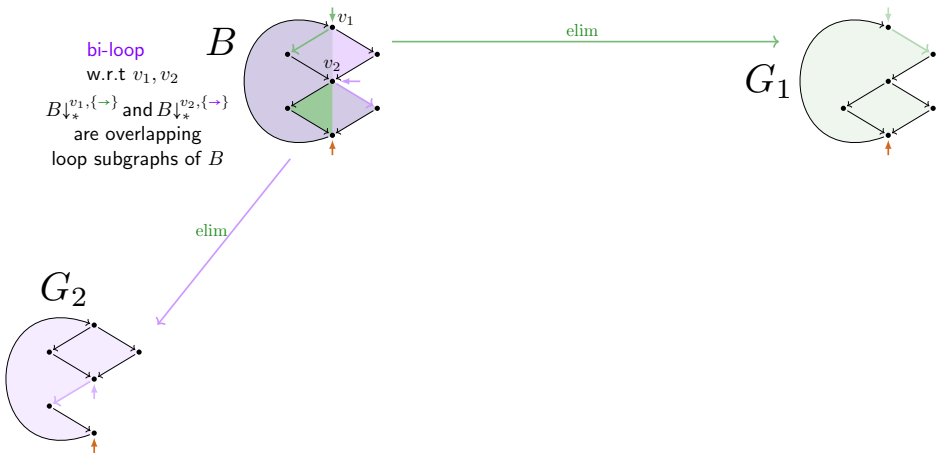
Non-Root overlap / 'critical pair' (example)



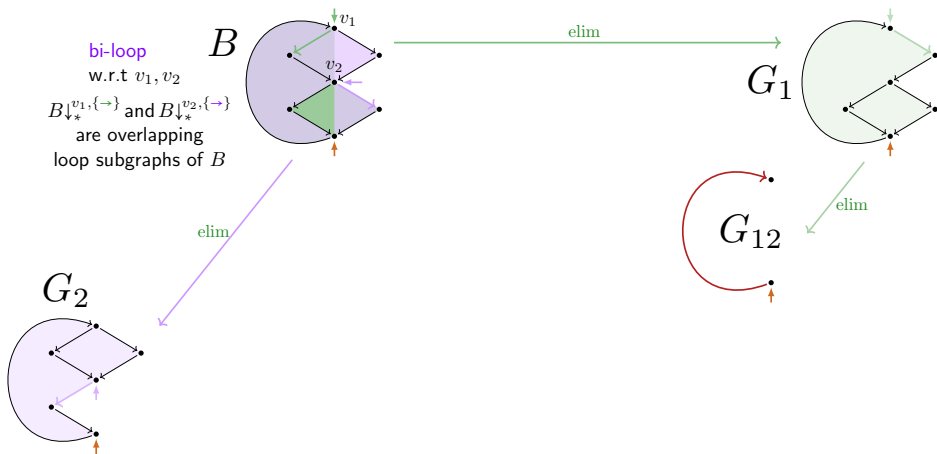
Non-Root overlap / 'critical pair' (example)



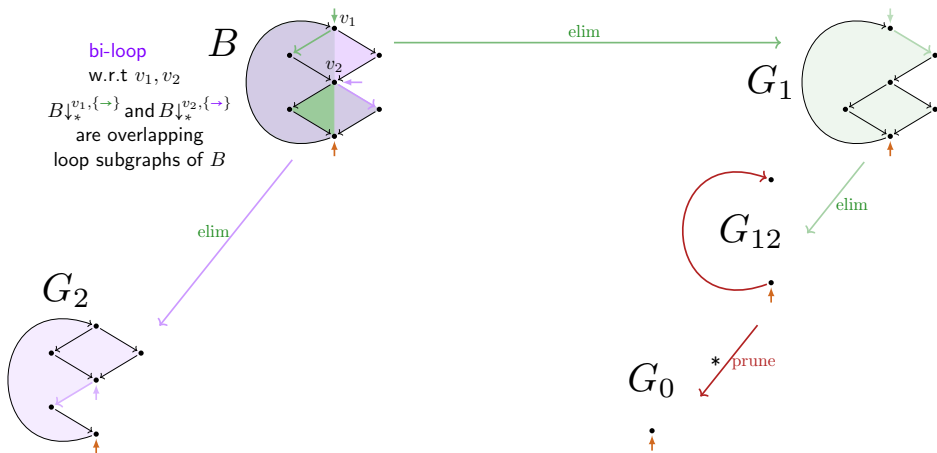
Non-Root overlap / 'critical pair' (example)



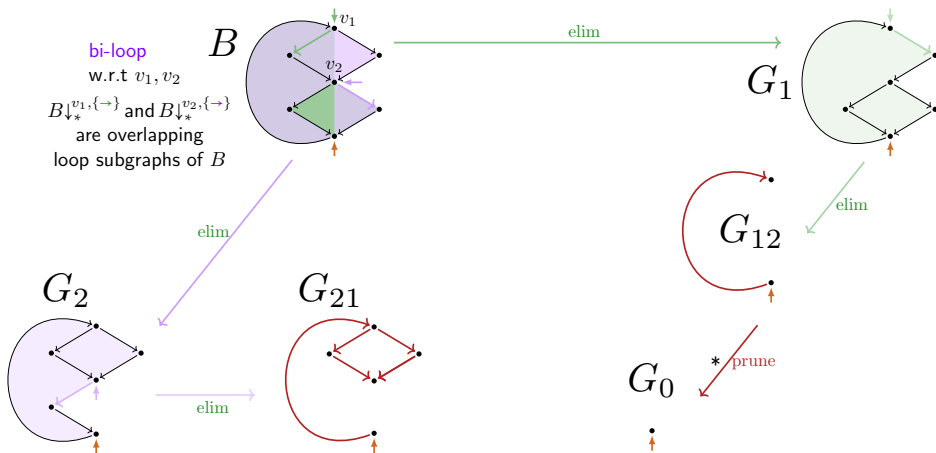
Non-Root overlap / 'critical pair' (example)



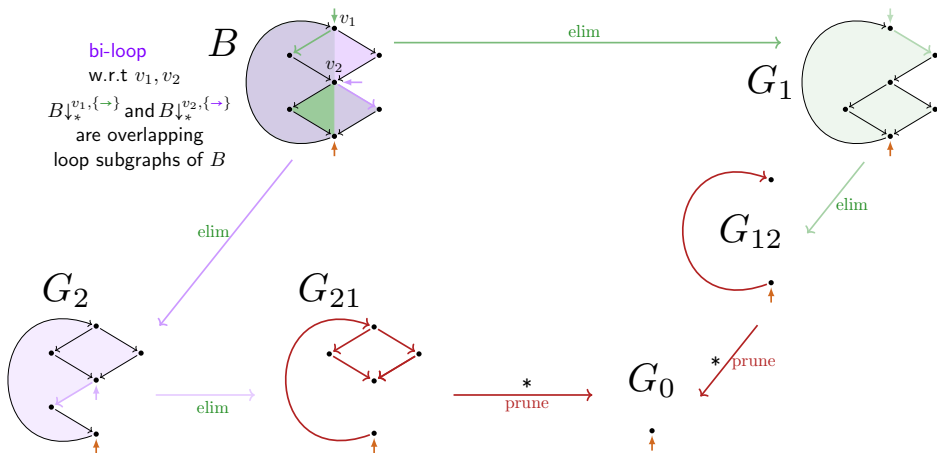
Non-Root overlap / 'critical pair' (example)



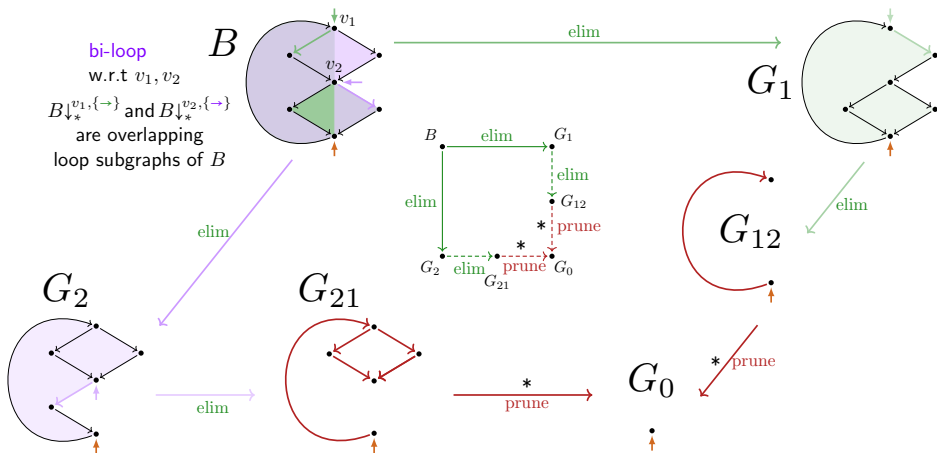
Non-Root overlap / 'critical pair' (example)



Non-Root overlap / 'critical pair' (example)



Non-Root overlap / 'critical pair' (example)



'Critical Pair Lemma'

Lemma (joining $\rightarrow_{\text{elim}}$ forks)

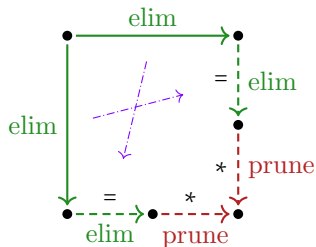
Forks $G_2 \leftarrow_{\text{elim}} G \rightarrow_{\text{elim}} G_1$ in which loop subgraphs $G_{\downarrow_*}^{v_2, T_2}$, $G_{\downarrow_*}^{v_1, T_1}$ of G are eliminated, can be joined with $\rightarrow_{\text{prune}}$ steps in the three situations:

- (i) **Root-overlap**: $v_1 = v_2$, and the loops form bigger loop $G_{\downarrow_*}^{v_1, T_1 \cup T_2}$.
The $\rightarrow_{\text{elim}}$ steps may overlap in their loop-entry transitions, but the fork can always be joined by eliminating residual loop-entries as follows: $G_2 \xrightarrow{=}_{\text{elim}} \cdot \xrightarrow{*}_{\text{prune}} G_0 \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=}_{\text{elim}} G_1$.
- (ii) **Parallel/nested**: $v_1 \neq v_2$, and the cyclic parts $\bigcirc(G_{\downarrow_*}^{v_1, T_1})$, $\bigcirc(G_{\downarrow_*}^{v_2, T_2})$ of the loops are vertex-disjoint. Then the $\rightarrow_{\text{elim}}$ steps are independent of each other, except that the elimination of one loop may eliminate the other in the garbage collection operation. The fork can always be joined as follows: $G_2 \xrightarrow{=}_{\text{elim}} G_0 \xleftarrow{=}_{\text{elim}} G_1$.
- (iii) **Non-root overlap**: $v_1 \neq v_2$, the start-vertex connected cyclic parts $\bigcirc(G_{\downarrow_*}^{v_1, T_1})$, and $\bigcirc(G_{\downarrow_*}^{v_2, T_2})$ have common vertices. The two loops together form a **bi-loop** w.r.t. v_1, v_2 . The fork can always be joined as follows: $G_2 \xrightarrow{=}_{\text{elim}} \cdot \xrightarrow{*}_{\text{prune}} G_0 \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=}_{\text{elim}} G_1$.

Elementary diagrams

Lemma

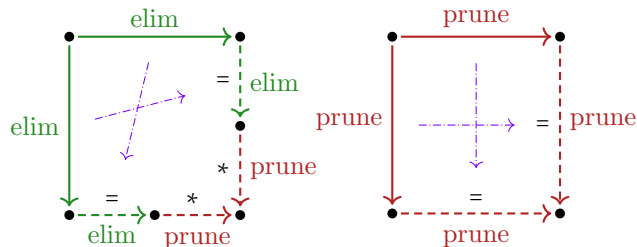
$\rightarrow_{ep}$ is (locally) decreasing with respect to $\{\rightarrow_\ell\}_{\ell \in \{\text{elim}, \text{prune}\}}$
for precedence $\text{prune} < \text{elim}$.



Elementary diagrams

Lemma

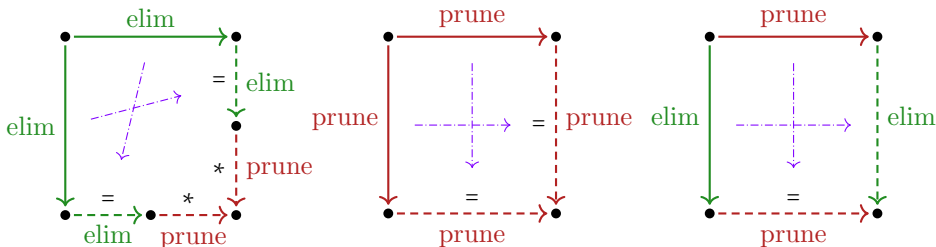
$\rightarrow_{ep}$ is (locally) decreasing with respect to $\{\rightarrow_\ell\}_{\ell \in \{\text{elim}, \text{prune}\}}$
for precedence $\text{prune} < \text{elim}$.



Elementary diagrams

Lemma

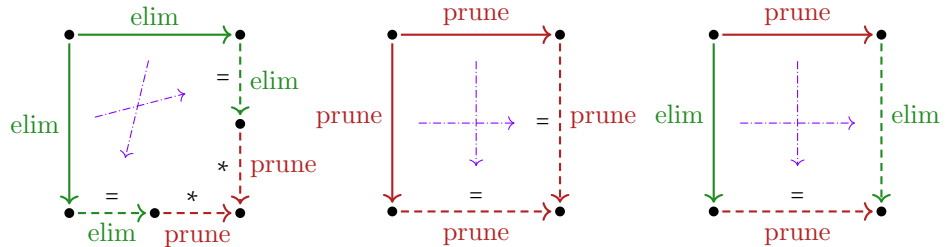
$\rightarrow_{ep}$ is (locally) decreasing with respect to $\{\rightarrow_\ell\}_{\ell \in \{\text{elim}, \text{prune}\}}$
for precedence $\text{prune} < \text{elim}$.



Elementary diagrams

Lemma

$\rightarrow_{ep}$ is (locally) decreasing with respect to $\{\rightarrow_\ell\}_{\ell \in \{\text{elim}, \text{prune}\}}$ for precedence $\text{prune} < \text{elim}$.



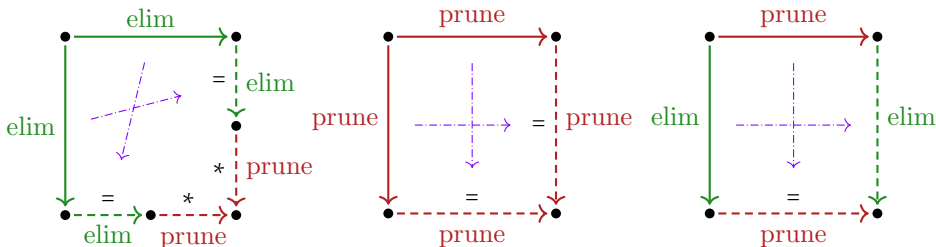
Theorem

$\rightarrow_{ep} = \rightarrow_{\text{elim}} \cup \rightarrow_{\text{prune}}$ is confluent.

Elementary diagrams

Lemma

$\rightarrow_{ep}$ is (locally) decreasing with respect to $\{\rightarrow_\ell\}_{\ell \in \{\text{elim}, \text{prune}\}}$ for precedence $\text{prune} < \text{elim}$.



Theorem

$\rightarrow_{ep} = \rightarrow_{\text{elim}} \cup \rightarrow_{\text{prune}}$ is confluent.

Proof.

By the Decreasing Diagrams Theorem (van Oostrom~'94/de Bruijn). $\square$

Overview

- ▶ Loop elimination $\rightarrow_{\text{elim}}$ in process graphs
 - ▶ Milner's process interpretation P of regular expressions
 - ▶ Loop existence and elimination property LEE
 - $\text{LEE} \hat{=}$ interpretations of $(*/\perp)$ reg. expr's
 - ▶ $\rightarrow_{\text{elim}}$ is **not** confluent
- ▶ **Completion** of loop elimination $\rightarrow_{\text{elim}}$ with **pruning steps** $\rightarrow_{\text{prune}}$
 - ▶ 'critical pair' lemma for 'bi-loop' elimination steps
 - ▶ use of decreasing diagrams
- ▶ **Application 1:** LEE is decidable in polynomial time
- ▶ **Application 2:** Layered LEE = LEE
- ▶ Further questions

Efficient decidability of LEE

Corollary (of confluence of $\rightarrow_{ep}$)

For every finite process graph G , TFAE:

- (i) $LEE(G)$: there is an $\rightarrow_{elim}$ normal form of G without infinite trace.
- (ii) The unique $\rightarrow_{ep}$ normal form of G does not enable an infinite trace.
- (iii) Every $\rightarrow_{elim}$ normal form of G does not enable an infinite trace.

Efficient decidability of LEE

Corollary (of confluence of $\rightarrow_{ep}$)

For every finite process graph G , TFAE:

- (i) $LEE(G)$: there is an $\rightarrow_{elim}$ normal form of G without infinite trace.
- (ii) The unique $\rightarrow_{ep}$ normal form of G does not enable an infinite trace.
- (iii) Every $\rightarrow_{elim}$ normal form of G does not enable an infinite trace.

Theorem

For finite graphs G , LEE is decidable in $\leq O(|G|^3) \in \text{PTIME}$.

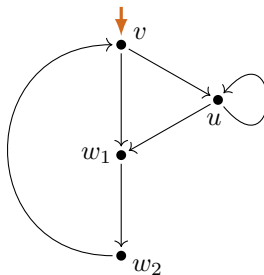
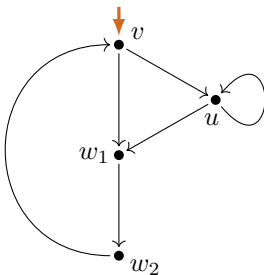
Proof

Due to (i) $\Rightarrow$ (iii) in the Corollary, to decide $LEE(G)$ it suffices to construct any $\rightarrow_{elim}$ rewrite sequence $G \rightarrow_{elim}^* G_0 \not\rightarrow_{elim}$ to normal form G_0 , and check if $\neg_{\infty}(G_0)$ holds. That is possible in $\leq O(|G|^3)$ time. $\square$

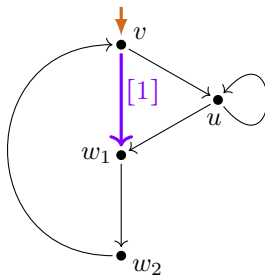
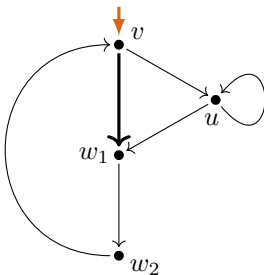
Overview

- ▶ Loop elimination $\rightarrow_{\text{elim}}$ in process graphs
 - ▶ Milner's process interpretation P of regular expressions
 - ▶ Loop existence and elimination property LEE
 - $\text{LEE} \hat{=}$ interpretations of $(*/\perp)$ reg. expr's
 - ▶ $\rightarrow_{\text{elim}}$ is **not** confluent
- ▶ **Completion** of loop elimination $\rightarrow_{\text{elim}}$ with **pruning steps** $\rightarrow_{\text{prune}}$
 - ▶ 'critical pair' lemma for 'bi-loop' elimination steps
 - ▶ use of decreasing diagrams
- ▶ **Application 1:** LEE is decidable in polynomial time
- ▶ **Application 2:** Layered LEE = LEE
- ▶ Further questions

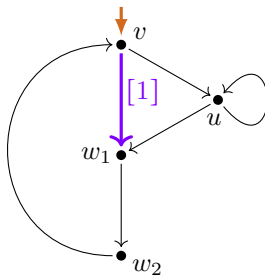
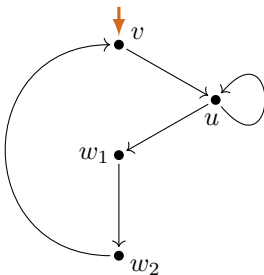
Not Layered / Layered Loop Elimination



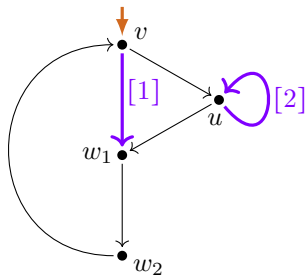
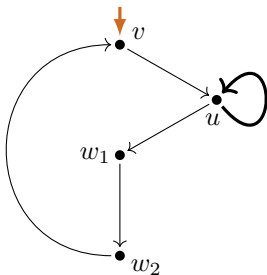
Not Layered / Layered Loop Elimination



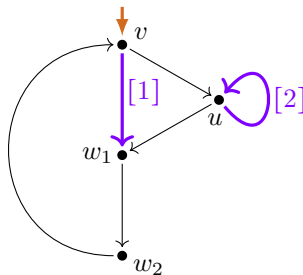
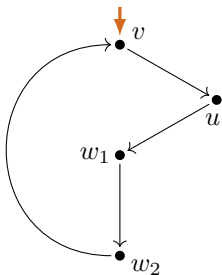
Not Layered / Layered Loop Elimination



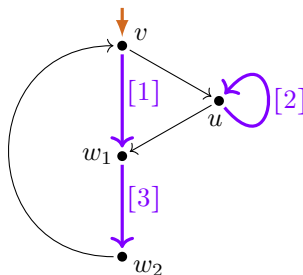
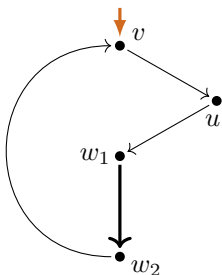
Not Layered / Layered Loop Elimination



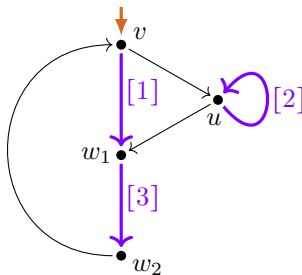
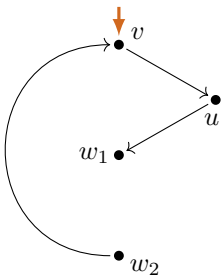
Not Layered / Layered Loop Elimination



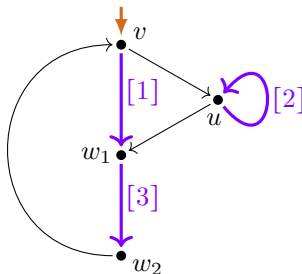
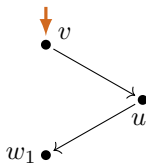
Not Layered / Layered Loop Elimination



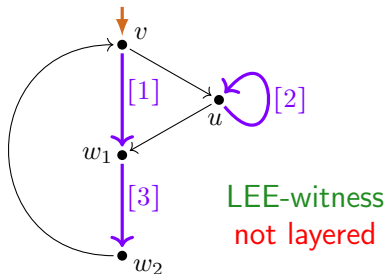
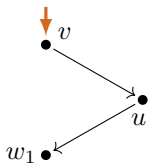
Not Layered / Layered Loop Elimination



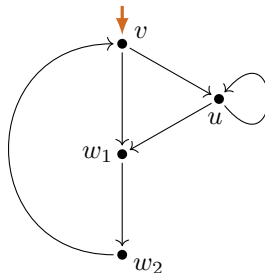
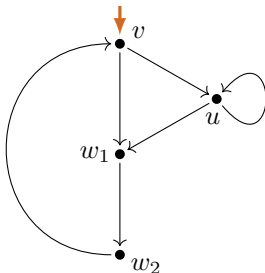
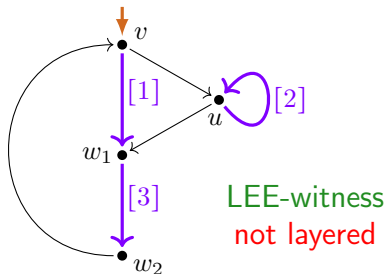
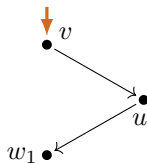
Not Layered / Layered Loop Elimination



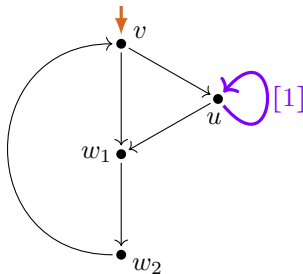
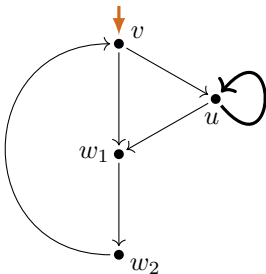
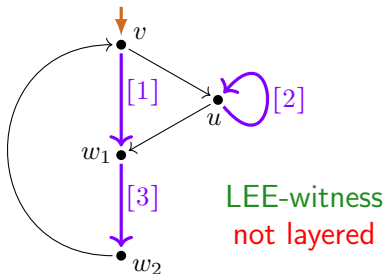
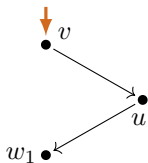
Not Layered / Layered Loop Elimination



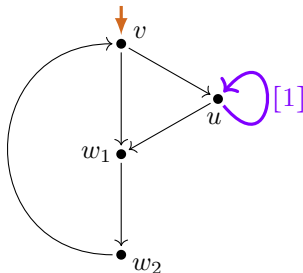
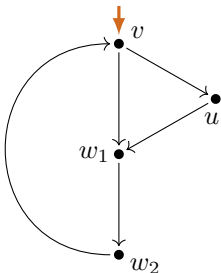
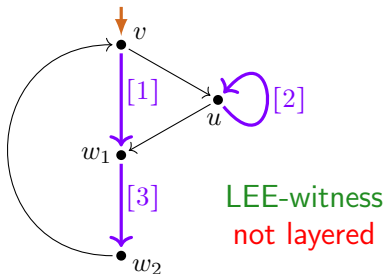
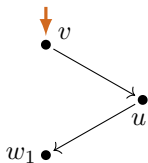
Not Layered / Layered Loop Elimination



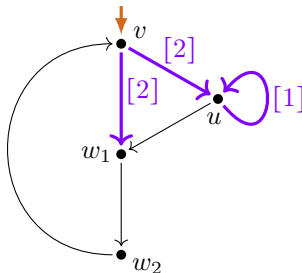
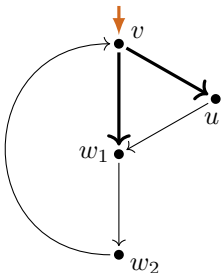
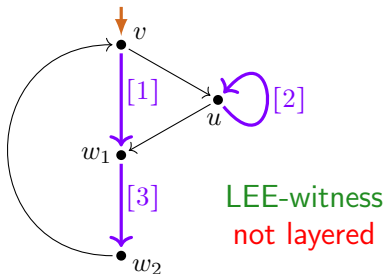
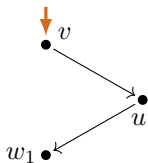
Not Layered / Layered Loop Elimination



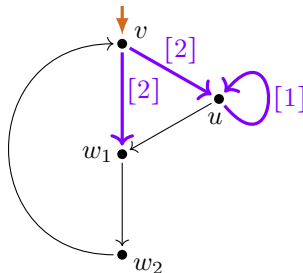
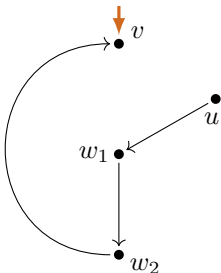
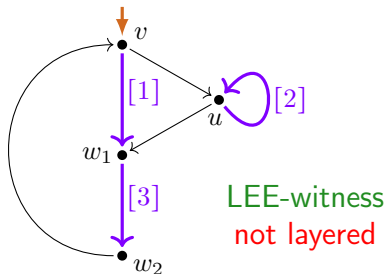
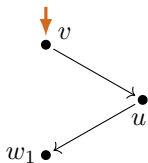
Not Layered / Layered Loop Elimination



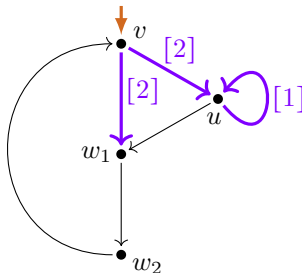
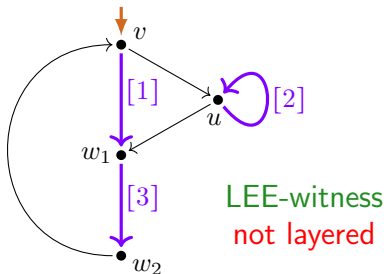
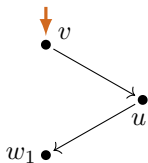
Not Layered / Layered Loop Elimination



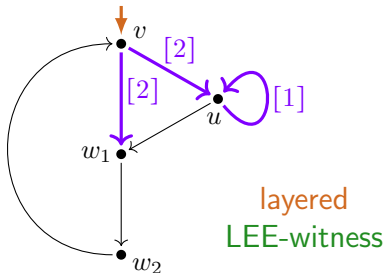
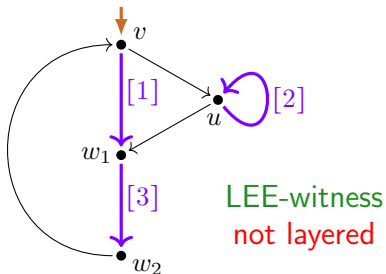
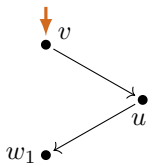
Not Layered / Layered Loop Elimination



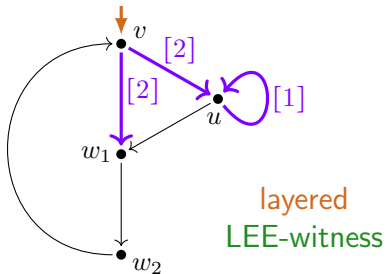
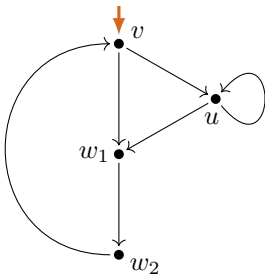
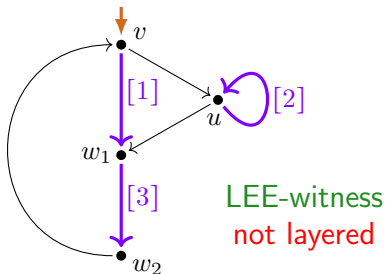
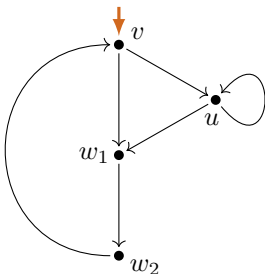
Not Layered / Layered Loop Elimination



Not Layered / Layered Loop Elimination



Not Layered / Layered Loop Elimination



Layered Loop Elimination Condition LLEE

Layered loop elimination is defined on **vertex-marked graphs** ${}^\circ G$ in order to:

- ▶ **mark** vertices in the **loop bodies** of **eliminated** loops,
- ▶ **permit** the elimination of loop subcharts **only at unmarked** vertices.

Definition

The **Layered Loop Existence & Elimination Condition LLEE** for graphs G :

$$\text{LLEE}(G) : \Longleftrightarrow \exists \bullet G_0 \left(({}^\circ G \xrightarrow[\text{elim}]{*} \bullet G_0 \xrightarrow[\text{elim}]{} \bullet G_0) \wedge \neg \infty(\bullet G_0) \right).$$

Layered Loop Elimination Condition LLEE

Layered loop elimination is defined on **vertex-marked graphs** $\circ G$ in order to:

- ▶ **mark** vertices in the **loop bodies** of **eliminated** loops,
- ▶ **permit** the elimination of loop subcharts **only at unmarked** vertices.

Definition

The **Layered Loop Existence & Elimination Condition LLEE** for graphs G :

$$\text{LLEE}(G) : \Longleftrightarrow \exists \bullet G_0 \left((\circ G \xrightarrow[\text{elim}]{*} \bullet G_0 \not\xrightarrow[\text{elim}]{}) \wedge \neg \infty(\bullet G_0) \right).$$

$$\bullet G_1 \xrightarrow[\text{elim}]{} \bullet G'_1 \implies |\bullet G_1|_{\circ} \xrightarrow[\text{elim}]{} |\bullet G'_1|_{\circ}$$

Layered Loop Elimination Condition LLEE

Layered loop elimination is defined on **vertex-marked graphs** ${}^\circ G$ in order to:

- ▶ **mark** vertices in the **loop bodies** of **eliminated** loops,
- ▶ **permit** the elimination of loop subcharts **only at unmarked** vertices.

Definition

The **Layered Loop Existence & Elimination Condition LLEE** for graphs G :

$$\text{LLEE}(G) : \Longleftrightarrow \exists \bullet G_0 \left(({}^\circ G \xrightarrow[\text{elim}]{*} \bullet G_0 \not\xrightarrow[\text{elim}]{\circ}) \wedge \neg \infty(\bullet G_0) \right).$$

$$\bullet G_1 \xrightarrow[\text{elim}]{\circ} \bullet G'_1 \implies |\bullet G_1|_{\circ} \xrightarrow[\text{elim}]{} |\bullet G'_1|_{\circ}$$

$${}^\circ G \xrightarrow[\text{elim}]{*} \bullet G_1 \implies \left[\exists \bullet G''_1 \left[\bullet G_1 \xrightarrow[\text{elim}]{\circ} \bullet G''_1 \wedge |\bullet G''_1|_{\circ} \xrightarrow[\text{prune}]{*} \bullet \xleftarrow[\text{prune}]{*} \bullet \xleftarrow[\text{elim}]{=} G'_1 \right] \Longleftrightarrow |\bullet G_1|_{\circ} \xrightarrow[\text{elim}]{} G'_1 \right]$$

Layered Loop Elimination Condition LLEE

Layered loop elimination is defined on **vertex-marked graphs** ${}^\circ G$ in order to:

- ▶ **mark** vertices in the **loop bodies** of **eliminated** loops,
- ▶ **permit** the elimination of loop subcharts **only at unmarked** vertices.

Definition

The **Layered Loop Existence & Elimination Condition LLEE** for graphs G :

$$\text{LLEE}(G) : \iff \exists {}^\bullet G_0 \left(({}^\circ G \xrightarrow{*}_{\text{elim}} {}^\bullet G_0 \not\xrightarrow{\circ}_{\text{elim}}) \wedge \neg \infty({}^\bullet G_0) \right).$$

$${}^\bullet G_1 \xrightarrow{\circ}_{\text{elim}} {}^\bullet G'_1 \implies |{}^\bullet G_1|_{\circ} \xrightarrow{\circ}_{\text{elim}} |{}^\bullet G'_1|_{\circ}$$

$${}^\circ G \xrightarrow{*}_{\text{elim}} {}^\bullet G_1 \implies \left[\exists {}^\bullet G''_1 \left[{}^\bullet G_1 \xrightarrow{\circ}_{\text{elim}} {}^\bullet G''_1 \wedge |{}^\bullet G''_1|_{\circ} \xrightarrow{*}_{\text{prune}} \cdot \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=} \text{elim} G'_1 \right] \iff |{}^\bullet G_1|_{\circ} \xrightarrow{\circ}_{\text{elim}} G'_1 \right]$$

$$\exists {}^\bullet G_1 \left[{}^\circ G \xrightarrow{\circ}_{\text{elim}} {}^\bullet G_1 \wedge |{}^\bullet G_1|_{\circ} \xleftrightarrow{*}_{\text{prune}} G_1 \right] \iff G \xrightarrow{*}_{\text{elim}} G_1$$

Layered Loop Elimination Condition LLEE

Layered loop elimination is defined on **vertex-marked graphs** ${}^\circ G$ in order to:

- ▶ **mark** vertices in the **loop bodies** of **eliminated** loops,
- ▶ **permit** the elimination of loop subcharts **only at unmarked** vertices.

Definition

The **Layered Loop Existence & Elimination Condition LLEE** for graphs G :

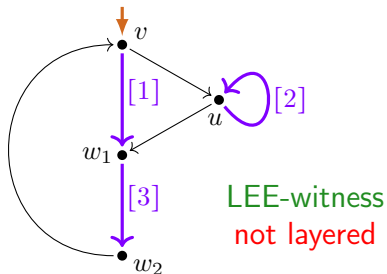
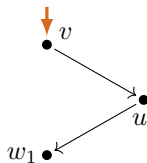
$$\text{LLEE}(G) : \Longleftrightarrow \exists \bullet G_0 \left(({}^\circ G \xrightarrow{*}_{\text{elim}} \bullet G_0 \not\xrightarrow{\circ}_{\text{elim}}) \wedge \neg \infty(\bullet G_0) \right).$$

$$\begin{aligned} \bullet G_1 \xrightarrow{\circ}_{\text{elim}} \bullet G'_1 &\implies |\bullet G_1|_{\circ} \xrightarrow{\circ}_{\text{elim}} |\bullet G'_1|_{\circ} \\ {}^\circ G \xrightarrow{*}_{\text{elim}} \bullet G_1 &\implies \left[\exists \bullet G''_1 \left[\bullet G_1 \xrightarrow{\circ}_{\text{elim}} \bullet G''_1 \wedge |\bullet G''_1|_{\circ} \xrightarrow{*}_{\text{prune}} \cdot \xleftarrow{*}_{\text{prune}} \cdot \xleftarrow{=} \text{elim} G'_1 \right] \Longleftrightarrow |\bullet G_1|_{\circ} \xrightarrow{\circ}_{\text{elim}} G'_1 \right] \\ \exists \bullet G_1 \left[{}^\circ G \xrightarrow{\circ}_{\text{elim}} \bullet G_1 \wedge |\bullet G_1|_{\circ} \xleftrightarrow{*}_{\text{prune}} G_1 \right] &\Longleftrightarrow G \xrightarrow{*}_{\text{elim}} G_1 \end{aligned}$$

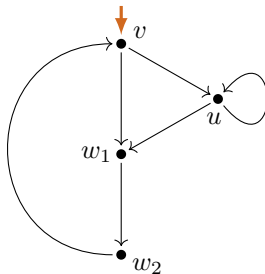
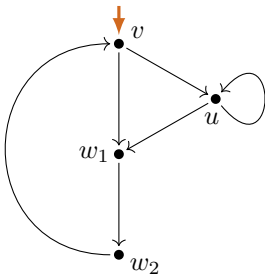
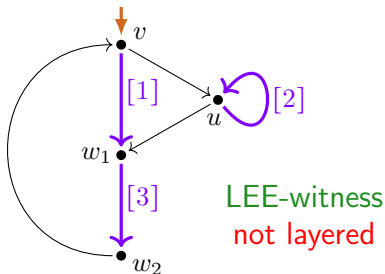
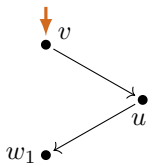
Theorem

$\text{LLEE}(G) \Leftrightarrow \text{LEE}(G)$ for all graphs G .

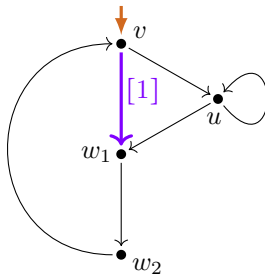
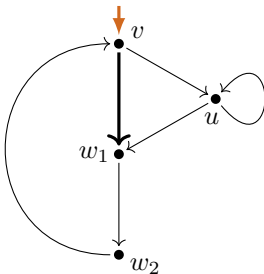
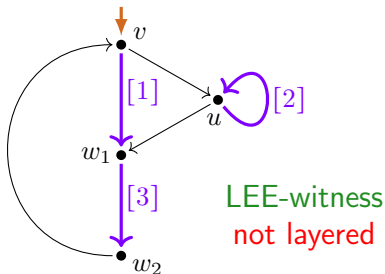
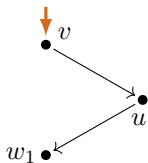
Making Layered



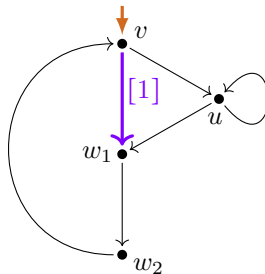
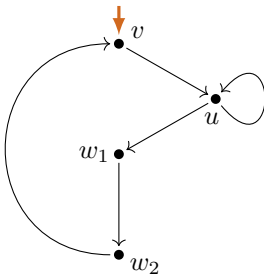
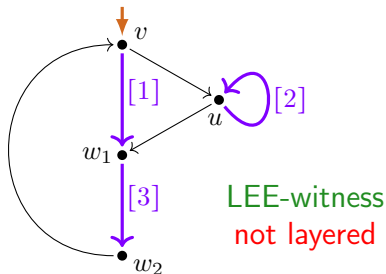
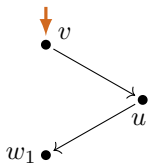
Making Layered



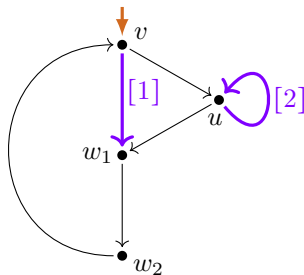
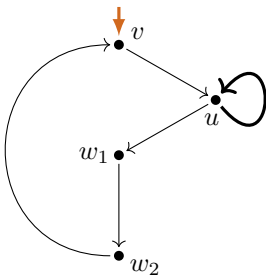
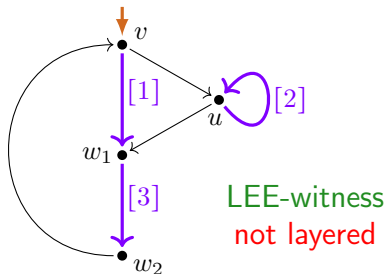
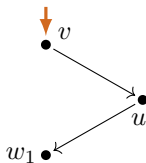
Making Layered



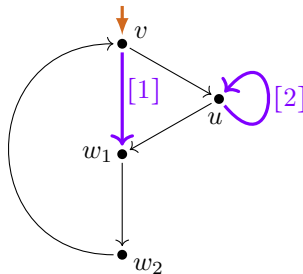
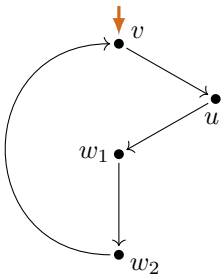
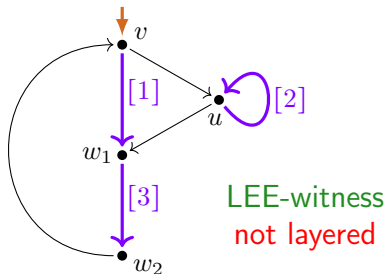
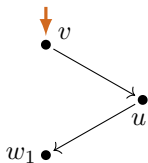
Making Layered



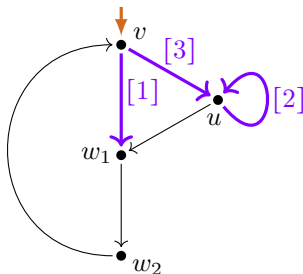
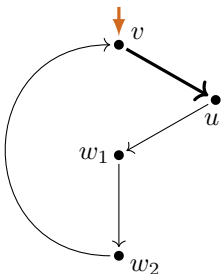
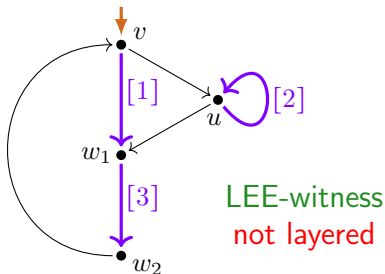
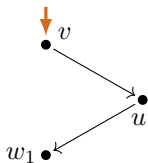
Making Layered



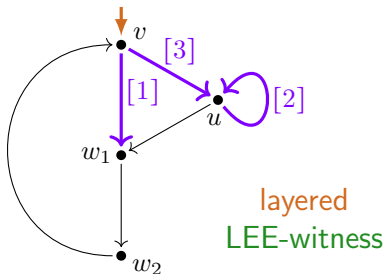
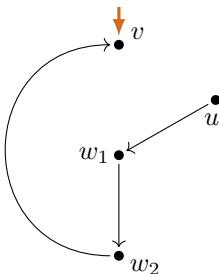
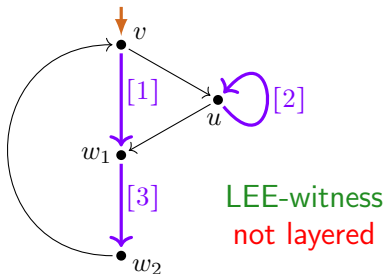
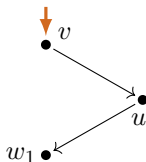
Making Layered



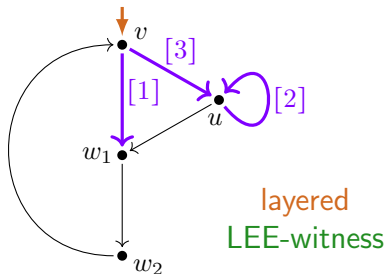
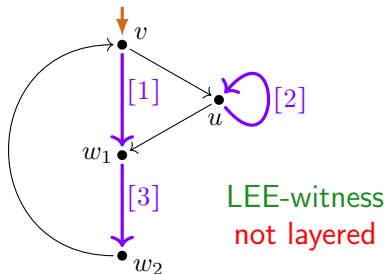
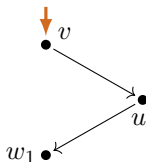
Making Layered



Making Layered



Making Layered



Further questions

(Q1) How efficient ($\leq O((n+k)^3)$) can LEE be decided?

- ▶ To try: characterisation of LEE as flow-graph co-reducibility (Laarakker–Kappé, TERMGRAPH 2026).

(Q2) How can, for a graph G with LEE, any two regular expressions e_1, e_2 with $P(e_1) = P(e_2) \simeq G$ that are extracted from maximal $\rightarrow_{\text{elim}}$ rewrite sequences from G be proved equivalent with respect to $\llbracket \cdot \rrbracket_P$?

(Q3) Can the Decreasing Diagrams Method be adapted to show pre-/post-ponement? (Here postponement of $\overset{o}{\rightarrow}_{\text{elim}}$ over $\rightarrow_{\text{elim}}$ is closely connected to confluence of $\rightarrow_{\text{ep}}$.)

- ▶ Vincent van Oostrom referred me to his IWC 2020 article.

Summary

- ▶ Loop elimination $\rightarrow_{\text{elim}}$ in process graphs
 - ▶ Milner's process interpretation P of regular expressions
 - ▶ Loop existence and elimination property LEE
 $\text{LEE} = \text{compact}$ process interpretations of $(*/\perp)$ reg. expr's
 - ▶ $\rightarrow_{\text{elim}}$ is **not** confluent
- ▶ **Completion** of loop elimination $\rightarrow_{\text{elim}}$ with **pruning steps** $\rightarrow_{\text{prune}}$
 - ▶ 'critical pair' lemma for joining elimination forks $\leftarrow_{\text{elim}} \cdot \rightarrow_{\text{elim}}$
 - ▶ use of decreasing diagrams for $\rightarrow_{\text{ep}} = \rightarrow_{\text{elim}} \cup \rightarrow_{\text{prune}}$
- ▶ **Application 1:** LEE is decidable in polynomial time
- ▶ **Application 2:** Layered LEE = LEE

IFIP-1.6 and TERMGRAPH talks

TERMGRAPH (last Sunday)

CG: *Towards a Characterisation of the Graph Structure of Process Interpretations of Regular Expressions*

Ext. Abstract <https://clegra.github.io/lf/pi-struc-TG-26.pdf>

Slides <https://clegra.github.io/lf/TG-26.pdf>

- ▶ $\text{LEE} \stackrel{\wedge}{=} \text{image of } P^\bullet|_{\text{RExp}(*/\perp)}$
- ▶ $1\text{-LEE} \stackrel{\wedge}{=} \text{image of } P^\bullet$

IFIP-1.6 and TERMGRAPH talks

TERMGRAPH (last Sunday)

CG: *Towards a Characterisation of the Graph Structure of Process Interpretations of Regular Expressions*

Ext. Abstract <https://clegra.github.io/lf/pi-struc-TG-26.pdf>

Slides <https://clegra.github.io/lf/TG-26.pdf>

- ▶ $\text{LEE} \stackrel{\wedge}{=} \text{image of } P^\bullet|_{\text{RExp}(*/\perp)}$
- ▶ $1\text{-LEE} \stackrel{\wedge}{=} \text{image of } P^\bullet$

IFIP-1.6 meeting (last Saturday)

CG: *Finding Small Steps for Hard Problems, in Work on the Process Interpretation of Regular Expressions*

Slides https://clegra.github.io/lf/IFIP-1_6-2026.pdf

- ▶ Reducing for Graphs Representing Regular Expressions under Bisimilarity

Resources

- ▶ Slides/extended abstract on clegra.github.io
 - ▶ slides: [.../1f/IWC-26.pdf](#)
 - ▶ extended abstract: [.../1f/1e-conf-IWC-26.pdf](#)
- ▶ CG: Milner's Proof System for Regular Expressions Modulo Bisimilarity is Complete
 - ▶ LICS 2022, [arXiv:2209.12188](#), poster.
- ▶ CG: The Image of the Process Interpretation of Regular Expressions is Not Closed under Bisimulation Collapse
 - ▶ [arXiv:2303.08553](#), 2021/2023.
- ▶ CG, Wan Fokkink: A Complete Proof System for 1-Free Regular Expressions Modulo Bisimilarity,
 - ▶ LICS 2020, [arXiv:2004.12740](#), video on youtube.
- ▶ CG: Structure-Constrained Process Graphs for the Process Semantics of Regular Expressions
 - ▶ TERMGRAPH 2020, [EPTCS 334](#), [arXiv:2012.10869](#).
- ▶ CG: Modeling Terms by Graphs with Structure Constraints
 - ▶ TERMGRAPH 2018, [EPTCS 288](#), [arXiv:1902.02010](#).